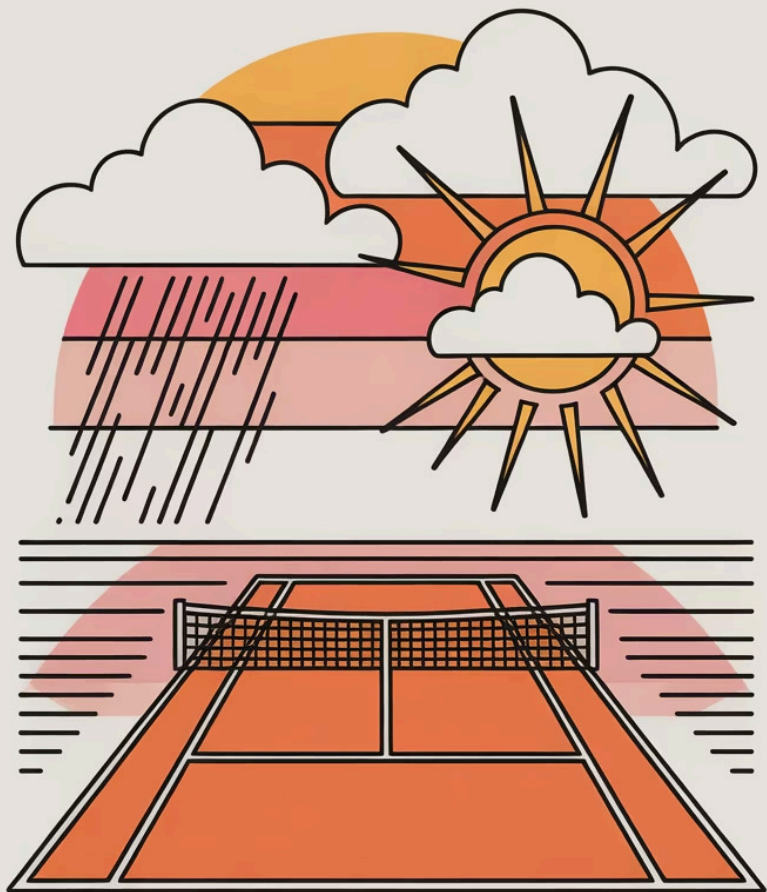




Understanding Decision Trees with the Play Tennis Dataset

A comprehensive guide to building decision trees from scratch, exploring entropy, information gain, and Gini impurity through a practical weather prediction example.

What is a Decision Tree?

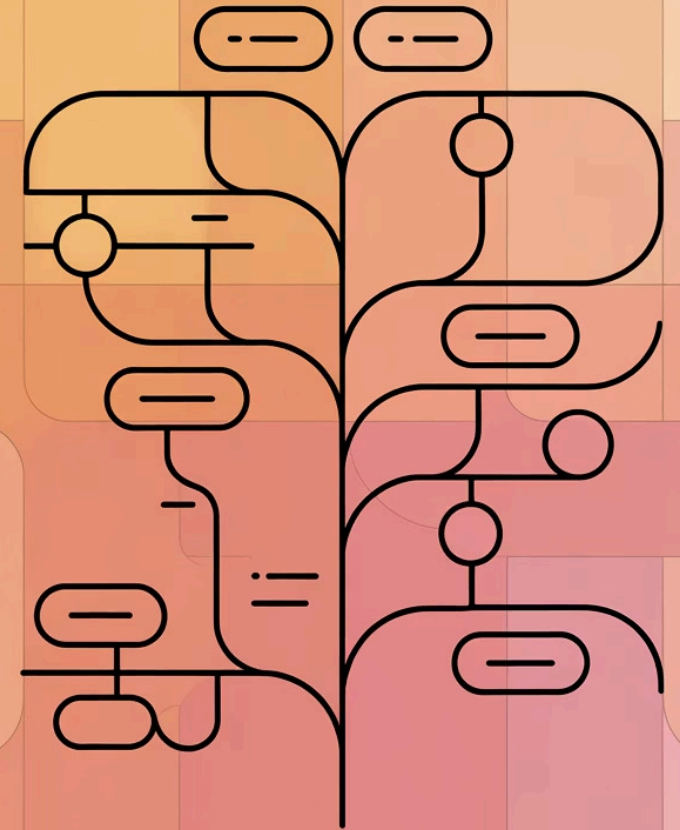
A decision tree is a flowchart-like structure that makes predictions by asking a series of questions about your data. It splits data based on feature values to predict outcomes, mimicking human "if-then-else" logic for classification tasks.

Think of it as a game of "20 questions" where each question narrows down the possibilities until you reach a final decision.

Real-World Example

Should we play tennis based on weather conditions? The tree learns patterns like: "If it's overcast, always play. If sunny, check humidity first."

This intuitive structure makes decision trees one of the most interpretable machine learning algorithms.



Meet the Play Tennis Dataset

Below is the complete dataset we will use to build our decision tree model, which consists of 14 days of weather observations and corresponding decisions to play tennis.

Day	Outlook	Temperature	Humidity	Wind	Play Tennis (target)
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No



Key Concept: Entropy — Measuring Uncertainty

Entropy quantifies the **uncertainty** or **impurity** in our dataset. It tells us how mixed our classes are before making a split.

Entropy Formula for Binary Classification

$$Entropy(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

where p_+ = proportion of positive (Yes) and p_- = proportion of negative (No)

What Does Entropy Really Mean?

Intuitively, entropy is a measure of **disorder** or **randomness**. Think of it like this:

- If a collection of items is perfectly organized (e.g., all "Yes" or all "No"), there's no surprise when you pick an item. The system has low entropy.
- If the items are completely mixed and equally likely (e.g., 50% "Yes" and 50% "No"), then picking an item is highly unpredictable. This system has high entropy.

In decision trees, we want to split data in a way that *reduces* entropy, moving from a state of high disorder to more organized, purer subsets.

Entropy in Different Scenarios

Perfect Purity (All Yes or All No)

If all samples belong to the same class (e.g., 100% "Yes", 0% "No"), there is no uncertainty.

Entropy = **0**

Maximum Uncertainty (50% Yes, 50% No)

If samples are split evenly between two classes, uncertainty is at its peak.

Entropy = **1.0**

Our Play Tennis Dataset (9 Yes, 5 No)

With 9 "Yes" and 5 "No" outcomes, there's significant uncertainty, but not maximum.

Entropy = **0.940**

Calculating Entropy for the Play Tennis Dataset

01

1. Count the Classes

In our dataset: 9 Yes, 5 No out of 14 total samples.

02

2. Calculate Proportions

The total number of samples is $N = 14$.

$$p_+ = \text{Number of Yes} / N = 9 / 14 = 0.642857$$

$$p_- = \text{Number of No} / N = 5 / 14 = 0.357143$$

03

3. Apply Entropy Formula (Detailed Calculation)

Recall the formula:

$$Entropy(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

First, calculate the log base 2 values:

$$\log_2(0.642857) \approx -0.639$$

$$\log_2(0.357143) \approx -1.484$$

Now substitute these into the formula:

$$Entropy = -(0.642857 \times -0.639) - (0.357143 \times -1.484)$$

$$Entropy = -(-0.4107) - (-0.5298)$$

$$Entropy = 0.4107 + 0.5298$$

$$Entropy \approx 0.940$$

Why Log Base 2?

We use $\log_2$ (logarithm base 2) because entropy in information theory is typically measured in **bits**. One bit represents the amount of information needed to answer a binary question (a question with two equally likely answers, like "yes" or "no"). For example, if we have a perfectly balanced dataset (50% Yes, 50% No), we need 1 bit of information to classify a randomly chosen sample. The $\log_2$ function helps us quantify this information content or uncertainty in terms of bits.

Interpretation: An entropy of **0.940 bits** indicates high uncertainty in our Play Tennis dataset. If all samples were the same class, entropy would be 0 (perfect purity). If evenly split 50-50, entropy would be 1.0 (maximum uncertainty).

Information Gain: Choosing the Best Feature to Split

Information Gain (IG) measures how much a feature reduces entropy. We calculate IG for each feature and choose the one with the **highest gain** for splitting.

Information Gain Formula

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

where S is the current dataset, A is the attribute (feature), $Values(A)$ are the possible values for attribute A , S_v is the subset of S for which attribute A has value v , and $|S_v|$ and $|S|$ are the number of samples in S_v and S respectively.

This simplifies to: IG = Entropy(before split) - Weighted Entropy(after split)

Calculating Information Gain for Each Feature

Our initial dataset (S) has 14 samples: 9 "Yes" and 5 "No". We previously calculated Entropy(S) = 0.940.

1. Feature: Outlook

We split the dataset based on the 'Outlook' feature, which has three possible values: Sunny, Overcast, and Rain.

1	2	3
Sunny Branch Total samples: 5 Class distribution: 2 Yes, 3 No Proportions: $p_+ = 2/5 = 0.4$, $p_- = 3/5 = 0.6$ Entropy(Sunny) $= -(0.4 \log_2 0.4) - (0.6 \log_2 0.6)$ $= -(0.4 \times -1.3219) - (0.6 \times -0.7369)$ $= 0.5288 + 0.4421 = \mathbf{0.971}$	Overcast Branch Total samples: 4 Class distribution: 4 Yes, 0 No Proportions: $p_+ = 4/4 = 1.0$, $p_- = 0/4 = 0.0$ Entropy(Overcast) $= -(1.0 \log_2 1.0) - (0.0 \log_2 0.0)$ $= -(1.0 \times 0) - (0.0 \times \text{undefined, treated as } 0) = \mathbf{0.0}$ (pure!)	Rain Branch Total samples: 5 Class distribution: 3 Yes, 2 No Proportions: $p_+ = 3/5 = 0.6$, $p_- = 2/5 = 0.4$ Entropy(Rain) $= -(0.6 \log_2 0.6) - (0.4 \log_2 0.4)$ $= -(0.6 \times -0.7369) - (0.4 \times -1.3219)$ $= 0.4421 + 0.5288 = \mathbf{0.971}$

Weighted Average Entropy (after split by Outlook)

We weigh the entropy of each branch by the proportion of samples it contains relative to the total dataset.

$$\left(\frac{5}{14} \times 0.971\right) + \left(\frac{4}{14} \times 0.0\right) + \left(\frac{5}{14} \times 0.971\right) = 0.3468 + 0 + 0.3468 = \mathbf{0.6936}$$

Information Gain (Outlook) = Entropy(S) - Weighted Entropy(Outlook)

IG(Outlook) = 0.940 - 0.6936 = 0.2464

2. Feature: Temperature

Next, we calculate the Information Gain for splitting by 'Temperature' (Hot, Mild, Cool).

1	2	3
Hot Branch Total samples: 4 Class distribution: 2 Yes, 2 No Proportions: $p_+ = 2/4 = 0.5$, $p_- = 2/4 = 0.5$ Entropy(Hot) $= -(0.5 \log_2 0.5) - (0.5 \log_2 0.5)$ $= -(0.5 \times -1.0) - (0.5 \times -1.0) = 0.5 + 0.5 = \mathbf{1.0}$	Mild Branch Total samples: 6 Class distribution: 4 Yes, 2 No Proportions: $p_+ = 4/6 = 0.667$, $p_- = 2/6 = 0.333$ Entropy(Mild) $= -(0.667 \log_2 0.667) - (0.333 \log_2 0.333)$ $= -(0.667 \times -0.5849) - (0.333 \times -1.5849)$ $= 0.3902 + 0.5277 = \mathbf{0.9179}$	Cool Branch Total samples: 4 Class distribution: 3 Yes, 1 No Proportions: $p_+ = 3/4 = 0.75$, $p_- = 1/4 = 0.25$ Entropy(Cool) $= -(0.75 \log_2 0.75) - (0.25 \log_2 0.25)$ $= -(0.75 \times -0.415) - (0.25 \times -2.0)$ $= 0.3112 + 0.5 = \mathbf{0.8112}$

Weighted Average Entropy (after split by Temperature)

$$\left(\frac{4}{14} \times 1.0\right) + \left(\frac{6}{14} \times 0.9179\right) + \left(\frac{4}{14} \times 0.8112\right) = 0.2857 + 0.3934 + 0.2317 = \mathbf{0.9108}$$

IG(Temperature) = 0.940 - 0.9108 = 0.0292

3. Feature: Humidity

Now for 'Humidity' (High, Normal).

1	2
High Branch Total samples: 7 Class distribution: 3 Yes, 4 No Proportions: $p_+ = 3/7 = 0.428$, $p_- = 4/7 = 0.572$ Entropy(High) $= -(0.428 \log_2 0.428) - (0.572 \log_2 0.572)$ $= -(0.428 \times -1.226) - (0.572 \times -0.804)$ $= 0.5246 + 0.4599 = \mathbf{0.9845}$	Normal Branch Total samples: 7 Class distribution: 6 Yes, 1 No Proportions: $p_+ = 6/7 = 0.857$, $p_- = 1/7 = 0.143$ Entropy(Normal) $= -(0.857 \log_2 0.857) - (0.143 \log_2 0.143)$ $= -(0.857 \times -0.222) - (0.143 \times -2.806)$ $= 0.1902 + 0.4011 = \mathbf{0.5913}$

Weighted Average Entropy (after split by Humidity)

$$\left(\frac{7}{14} \times 0.9845\right) + \left(\frac{7}{14} \times 0.5913\right) = 0.4922 + 0.2956 = \mathbf{0.7878}$$

IG(Humidity) = 0.940 - 0.7878 = 0.1522

4. Feature: Wind

Finally, the 'Wind' feature (Weak, Strong).

1	2
Weak Branch Total samples: 8 Class distribution: 6 Yes, 2 No Proportions: $p_+ = 6/8 = 0.75$, $p_- = 2/8 = 0.25$ Entropy(Weak) $= -(0.75 \log_2 0.75) - (0.25 \log_2 0.25)$ $= -(0.75 \times -0.415) - (0.25 \times -2.0)$ $= 0.3112 + 0.5 = \mathbf{0.8112}$	Strong Branch Total samples: 6 Class distribution: 3 Yes, 3 No Proportions: $p_+ = 3/6 = 0.5$, $p_- = 3/6 = 0.5$ Entropy(Strong) $= -(0.5 \log_2 0.5) - (0.5 \log_2 0.5)$ $= -(0.5 \times -1.0) - (0.5 \times -1.0) = 0.5 + 0.5 = \mathbf{1.0}$

Weighted Average Entropy (after split by Wind)

$$\left(\frac{8}{14} \times 0.8112\right) + \left(\frac{6}{14} \times 1.0\right) = 0.4635 + 0.4286 = \mathbf{0.8921}$$

IG(Wind) = 0.940 - 0.8921 = 0.0479

Understanding Weighted Average Entropy

The "weighted average entropy after split" is crucial because it accounts for the fact that some branches resulting from a split might contain more data points than others. Each branch's entropy is multiplied by the proportion of the total dataset it represents. This ensures that larger, more influential branches have a greater impact on the overall entropy reduction, giving a fair measure of the feature's ability to purify the dataset.

Comparison of Information Gain Values

To determine the best feature for the first split (root node), we compare the Information Gain values for all features:

Feature	Information Gain
Outlook	0.2464
Temperature	0.0292
Humidity	0.1522
Wind	0.0479

As you can see, 'Outlook' has the highest Information Gain (0.2464). This means that splitting the dataset by 'Outlook' will result in the greatest reduction in entropy, leading to the purest possible subsets at this stage. Therefore, 'Outlook' is chosen as the root node for our decision tree.

Building the Complete Tree: Step-by-Step Walkthrough

This card shows the COMPLETE process of building the entire decision tree from the Play Tennis dataset.

Step 1: Root Node Selection

From our previous calculations, 'Outlook' has the highest Information Gain (0.2464) among all features. Therefore, 'Outlook' is chosen as the root node for our decision tree.

1

Root Node
Outlook

This split creates three branches: Sunny (5 samples), Overcast (4 samples), and Rain (5 samples).

Step 2: Overcast Branch



Overcast (4 Samples)
Class distribution: 4 Yes, 0 No
This branch is pure (all 'Yes').

Since the 'Overcast' branch is pure (all 4 samples result in 'Yes' for playing tennis), it forms a leaf node predicting 'Yes'. No further splitting is needed for this branch.

Leaf Node: Yes
Play Tennis = Yes

Step 3: Sunny Branch



Sunny (5 Samples)
Class distribution: 2 Yes, 3 No
This branch is impure and requires further splitting.

For the 5 samples where 'Outlook' is 'Sunny', the class distribution is 2 'Yes' and 3 'No'. We need to calculate Information Gain for the remaining features (Temperature, Humidity, Wind) on this subset only.

Information Gain for 'Sunny' subset:

- IG(Temperature|Sunny) = 0.0202
- IG(Humidity|Sunny) = 0.971
- IG(Wind|Sunny) = 0.0

The feature 'Humidity' has the highest Information Gain (0.971) for the 'Sunny' subset. Thus, we split the 'Sunny' branch by 'Humidity'.

1

Sunny -> Humidity
Split the 5 samples by 'Humidity':

- High Humidity:** 3 samples (0 Yes, 3 No) - This branch is pure ('No').
- Normal Humidity:** 2 samples (2 Yes, 0 No) - This branch is pure ('Yes').

This results in two new leaf nodes:

Leaf Node: No
If Outlook=Sunny AND Humidity=High, Play Tennis = No

Leaf Node: Yes
If Outlook=Sunny AND Humidity=Normal, Play Tennis = Yes

Step 4: Rain Branch



Rain (5 Samples)
Class distribution: 3 Yes, 2 No
This branch is impure and requires further splitting.

For the 5 samples where 'Outlook' is 'Rain', the class distribution is 3 'Yes' and 2 'No'. We need to calculate Information Gain for the remaining features (Temperature, Humidity, Wind) on this subset only.

Information Gain for 'Rain' subset:

- IG(Temperature|Rain) = 0.02
- IG(Humidity|Rain) = 0.02
- IG(Wind|Rain) = 0.971

The feature 'Wind' has the highest Information Gain (0.971) for the 'Rain' subset. Thus, we split the 'Rain' branch by 'Wind'.

1

Rain -> Wind
Split the 5 samples by 'Wind':

- Weak Wind:** 3 samples (3 Yes, 0 No) - This branch is pure ('Yes').
- Strong Wind:** 2 samples (0 Yes, 2 No) - This branch is pure ('No').

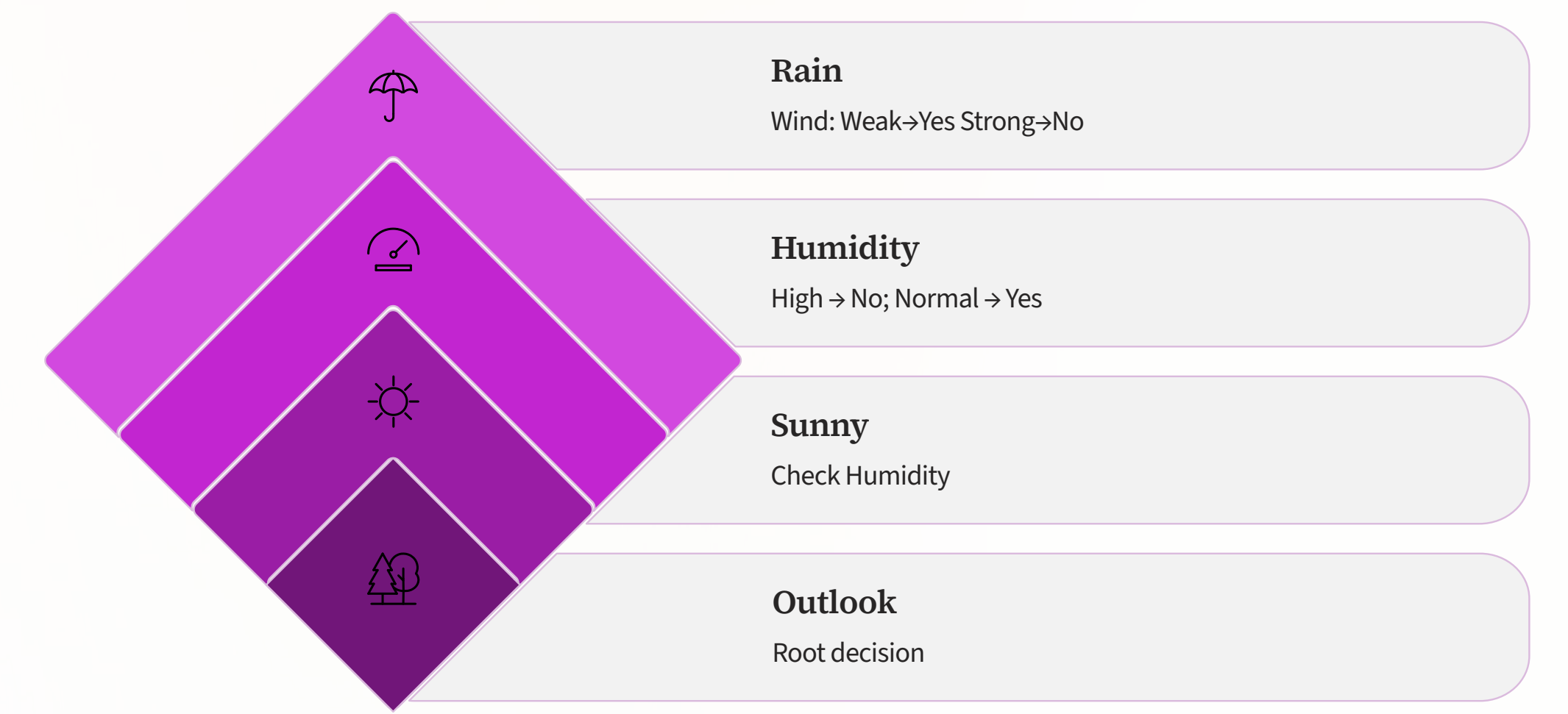
This results in two new leaf nodes:

Leaf Node: Yes
If Outlook=Rain AND Wind=Weak, Play Tennis = Yes

Leaf Node: No
If Outlook=Rain AND Wind=Strong, Play Tennis = No

Final Complete Tree Structure

Putting all the branches and leaf nodes together, here is the complete decision tree:



Gini Impurity: Another Measure of Purity

While ID3 and C4.5 algorithms use entropy, CART (Classification and Regression Trees) uses **Gini impurity** as an alternative splitting criterion.

Gini Formula

$$Gini(S) = 1 - p_+^2 - p_-^2$$

Gini ranges from 0 (pure) to 0.5 (maximum impurity for binary classification). It's computationally faster than entropy since it avoids logarithms.

Calculating Gini for Our Dataset

Using our proportions ($p_+ = 0.642$, $p_- = 0.357$):

$$Gini = 1 - (0.642)^2 - (0.357)^2 = 0.459$$

Lower Gini = Purer Node

- Gini = 0: Perfect purity (one class)
- Gini = 0.5: Maximum impurity (50-50 split)

Entropy vs Gini

Both measure impurity similarly. Entropy is theoretically motivated (information theory), while Gini is computationally simpler. In practice, they often produce similar trees.

Step-by-Step Building of the Decision Tree



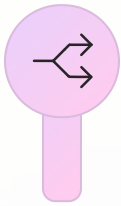
Calculate Root Entropy

Start with entire dataset: Entropy = 0.940 bits of uncertainty



Evaluate All Features

Calculate IG for Outlook, Temperature, Humidity, Wind. Outlook has highest IG = 0.247



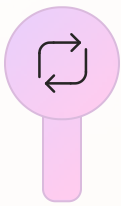
Create Root Split

Split dataset by Outlook into three branches: Sunny, Overcast, Rain



Check Branch Purity

Overcast branch is pure (all Yes) → Create leaf node, stop splitting



Recursively Split Impure Branches

For Sunny and Rain: Calculate IG for remaining features, split on best one



Build Complete Tree

Continue until all branches are pure or stopping criteria met

Partial Tree Structure

- **Root:** Outlook?
 - If Overcast → **Play Tennis = Yes** (pure leaf)
 - If Sunny → Split on Humidity (High → No, Normal → Yes)
 - If Rain → Split on Wind (Strong → No, Weak → Yes)

Detailed Calculations: Rain Branch Split

This card provides in-depth calculations for splitting the Rain branch (completing the recursive process demonstration).

After Root Split: Analyzing the Rain Branch

We have 5 samples where Outlook=Rain

Day	Temperature	Humidity	Wind	Play Tennis
4	Mild	High	Weak	Yes
5	Cool	Normal	Weak	Yes
6	Cool	High	Strong	No
10	Mild	Normal	Weak	Yes
14	Mild	High	Strong	No

Current entropy of Rain branch: 0.971 (3 Yes, 2 No)

Candidate Feature 1: Temperature

- Mild subset: Days 4, 10, 14 (2 Yes, 1 No) → Entropy = 0.918
- Cool subset: Days 5, 6 (1 Yes, 1 No) → Entropy = 1.0
- Weighted entropy = (3/5)×0.918 + (2/5)×1.0 = 0.951
- IG(Temperature|Rain) = 0.971 - 0.951 = 0.020

Candidate Feature 2: Humidity

- High subset: Days 4, 14 (1 Yes, 1 No) → Entropy = 1.0
- Normal subset: Days 5, 6, 10 (2 Yes, 1 No) → Entropy = 0.918
- Weighted entropy = (2/5)×1.0 + (3/5)×0.918 = 0.951
- IG(Humidity|Rain) = 0.971 - 0.951 = 0.020

Candidate Feature 3: Wind

- Weak subset: Days 4, 5, 10 (3 Yes, 0 No) → Entropy = 0
- Strong subset: Days 6, 14 (0 Yes, 2 No) → Entropy = 0
- Weighted entropy = (3/5)×0 + (2/5)×0 = 0
- IG(Wind|Rain) = 0.971 - 0 = **0.971 ← HIGHEST!**

Decision: Split Rain on Wind

Wind gives perfect separation, creating two pure leaf nodes.

Tree Complete!

All branches are now pure, algorithm terminates.

Stopping Criteria: When to Stop Splitting

While our Play Tennis example stopped when branches became pure, real-world decision trees use multiple stopping criteria to prevent overfitting and control tree complexity.

Node Purity (Entropy/Gini = 0)

- Stop when all samples in a node belong to the same class
- This is what happened in our Play Tennis example
- Example: Overcast branch had 4 Yes, 0 No → pure, stop splitting

min_samples_split

- Minimum number of samples required to split an internal node
- Default in sklearn: 2
- Example: If min_samples_split=5, a node with only 3 samples cannot be split further, even if impure
- Prevents creating splits on very small subsets that might be noise

min_samples_leaf

- Minimum number of samples required to be at a leaf node
- Default in sklearn: 1
- Example: If min_samples_leaf=2, any split that would create a leaf with only 1 sample is rejected
- Ensures each decision has sufficient support

max_depth

- Maximum depth of the tree (number of splits from root to leaf)
- Default in sklearn: None (unlimited)
- Example: max_depth=3 means maximum 3 levels of splits
- Our Play Tennis tree has depth=2

max_leaf_nodes

- Maximum number of leaf nodes allowed
- Limits tree complexity by capping total number of decisions
- Example: max_leaf_nodes=10 stops growing when 10 leaves exist

min_impurity_decrease

- Minimum impurity decrease required to make a split
- A split is only made if it decreases impurity by at least this amount
- Example: min_impurity_decrease=0.01 means IG must be ≥ 0.01
- Prevents splits that barely improve purity

Example: Applying Stopping Criteria to Play Tennis Dataset

- min_samples_split=3:** Sunny and Rain branches (5 samples each) would still split, but if we had a branch with only 2 samples, it would stop
- max_depth=1:** Tree would only have root split on Outlook, no further splits on Humidity or Wind
- min_samples_leaf=3:** Some splits might be rejected if they create leaves with <3 samples

Comparison Table: Stopping Criteria Parameters

Parameter	Default	Effect of Increasing	Effect of Decreasing
min_samples_split	2	More pruning, less complex tree, higher bias	Less pruning, more complex tree, lower bias
min_samples_leaf	1	More pruning, less complex tree, higher bias	Less pruning, more complex tree, lower bias
max_depth	None	More pruning, less complex tree, higher bias	Less pruning, more complex tree, lower bias
max_leaf_nodes	None	More pruning, less complex tree, higher bias	Less pruning, more complex tree, lower bias
min_impurity_decrease	0.0	More pruning, less complex tree, higher bias	Less pruning, more complex tree, lower bias

In practice, you tune these hyperparameters using cross-validation to find the right balance between model complexity and generalization. Our simple Play Tennis example only needed purity checks, but real datasets require careful tuning of all these criteria.

Can Features Be Reused in Different Branches?

This card clarifies an important question: Once a feature is used for splitting, can it be used again in the tree?

The Short Answer: It Depends on the Branch!

Key Rule: Features CAN be reused in DIFFERENT branches

- Once you split on a feature (e.g., Outlook at root), that feature is NOT available for further splits in the SAME path
- BUT it CAN be used again in a DIFFERENT branch of the tree
- Think of it as: each path from root to leaf has its own independent feature selection

Example from Our Play Tennis Tree:

Show the tree structure:

- Root: Split on **Outlook** (creates 3 branches: Sunny, Overcast, Rain)
 - If Overcast → Pure, no further split needed
 - If Sunny branch → Split on **Humidity** (Outlook already used in this path)
 - If Rain branch → Split on **Wind** (Outlook already used in this path)

Important observation:

- In the Sunny path: Outlook → Humidity (we could NOT use Outlook again here)
- In the Rain path: Outlook → Wind (we could NOT use Outlook again here)
- But Humidity was used in Sunny branch and NOT used in Rain branch
- Similarly, Wind was used in Rain branch and NOT used in Sunny branch

Hypothetical Example: Feature Reuse in Different Branches

Imagine a different dataset where:

Root: Split on Temperature

- |—— Hot branch → Split on Temperature again? NO! (already in this path)
 - | |—— Split on Humidity? YES! (not used in this path yet)
- |—— Cold branch → Split on Temperature again? NO! (already in this path)
 - | |—— Split on Humidity? YES! (not used in this path yet)

But consider this scenario:

Root: Split on Outlook

- |—— Sunny → Split on Temperature
 - | |—— Hot → Split on Outlook again? NO! (already used in Sunny path)
 - | |—— Cold → Split on Outlook again? NO! (already used in Sunny path)
- |—— Rain → Split on Temperature? YES! (not used in Rain path yet)
 - | |—— Hot → Split on Outlook again? NO! (already used in Rain path)
 - | |—— Cold → Split on Temperature again? NO! (already used in Rain path)

Why This Rule Exists:

- Once you split on a feature in a path, all samples in that branch have already been separated by that feature's values
- Splitting on the same feature again in the same path would be redundant and provide no additional information
- However, different branches represent different subsets of data, so the same feature might be informative in a different context

Practical Implication:

- Each path from root to leaf uses a SUBSET of available features
- Different paths can use different combinations of features
- A feature used early in one branch might not be used at all in another branch
- This is why decision trees are flexible and can capture complex decision boundaries

Summary Box:

 **Feature Reuse Rule:** A feature can be used multiple times in a tree, but NOT in the same path from root to leaf. Each branch independently selects the best available features that haven't been used yet in that specific path.

Detailed Calculations: Sunny Branch Split

This card provides in-depth calculations for splitting the Sunny branch (to show the recursive process in detail).

After Root Split: Analyzing the Sunny Branch

We have 5 samples where Outlook=Sunny

Day	Temperature	Humidity	Wind	Play Tennis
1	Hot	High	Weak	No
2	Hot	High	Strong	No
8	Mild	High	Weak	No
9	Cool	Normal	Weak	Yes
11	Mild	Normal	Strong	Yes

Current entropy of Sunny branch: 0.971 (2 Yes, 3 No)

Candidate Feature 1: Temperature

- Hot subset: Days 1, 2 (0 Yes, 2 No) → Entropy = 0
- Mild subset: Days 8, 11 (1 Yes, 1 No) → Entropy = 1.0
- Cool subset: Day 9 (1 Yes, 0 No) → Entropy = 0

Weighted entropy = (2/5)×0 + (2/5)×1.0 + (1/5)×0 = 0.4

IG(Temperature|Sunny) = 0.971 - 0.4 = 0.571

Candidate Feature 2: Humidity

- High subset: Days 1, 2, 8 (0 Yes, 3 No) → Entropy = 0
- Normal subset: Days 9, 11 (2 Yes, 0 No) → Entropy = 0

Weighted entropy = (3/5)×0 + (2/5)×0 = 0

IG(Humidity|Sunny) = 0.971 - 0 = **0.971 ← HIGHEST!**

Candidate Feature 3: Wind

Calculate similarly and show it has lower IG than Humidity

Decision: Split Sunny on Humidity

Humidity gives perfect separation, creating two pure leaf nodes

The Complete Decision Tree: Visual Representation

This card provides a clear visual representation of the final decision tree with all decision rules, demonstrating how it classifies new data points.

Decision Rules Extracted

- 1. IF Outlook=Sunny AND Humidity=High THEN Play=No
- 2. IF Outlook=Sunny AND Humidity=Normal THEN Play=Yes
- 3. IF Outlook=Overcast THEN Play=Yes
- 4. IF Outlook=Rain AND Wind=Weak THEN Play=Yes
- 5. IF Outlook=Rain AND Wind=Strong THEN Play=No

Testing the Tree

Let's traverse the tree with a few examples to see how predictions are made.

Consider a new day with **Outlook=Sunny**, **Temperature=Hot**, **Humidity=High**, and **Wind=Strong**.

Following the tree: **Outlook: Sunny** → **Humidity: High** → **Play = No**.

For a day with **Outlook=Overcast**, **Temperature=Mild**, **Humidity=Normal**, and **Wind=Weak**.

Following the tree: **Outlook: Overcast** → **Play = Yes**.

And for a day with **Outlook=Rain**, **Temperature=Cool**, **Humidity=Normal**, and **Wind=Weak**.

Following the tree: **Outlook: Rain** → **Wind: Weak** → **Play = Yes**.

Tree Statistics

Tree depth = 2 (maximum number of splits to reach a leaf node)

Number of leaf nodes = 5

Features used = 2 out of 4 (Outlook, Humidity, Wind)

Where Decision Trees Can Fail

Overfitting

Trees grow too deep, memorizing training data noise. Result: poor performance on new data. Solution: Pruning, max depth limits, minimum samples per leaf.

Instability

Small changes in training data can produce completely different trees (high variance). Solution: Ensemble methods like Random Forests.

Categorical Bias

Features with many levels are favored during splitting, even if less informative. Solution: Use gain ratio (C4.5) or multiway vs binary splits.

Poor Extrapolation

Trees make piecewise constant predictions, can't extrapolate beyond training range. Problematic for regression with trends.

Complex Relationships

Struggle with XOR-like patterns or oblique decision boundaries. Linear combinations of features are difficult without ensembles.

Despite these limitations, decision trees remain powerful tools—especially when combined with ensemble methods or when interpretability is crucial.

Summary: How Decision Trees Work

1. Measure Impurity

Use entropy or Gini to quantify node uncertainty. Lower values indicate purer nodes with fewer mixed classes.

2. Choose Best Splits

Calculate information gain or Gini reduction for all features. Select the feature that maximizes purity improvement.

3. Recursive Splitting

Apply the same process to each branch until reaching stopping criteria: pure nodes, max depth, or minimum samples.

4. Build Interpretable Model

Result is a tree structure that mimics human decision-making with clear if-then rules.

The Play Tennis Example in Action

Our weather dataset demonstrated every step: calculating entropy (0.940), finding best split (Outlook with $IG=0.247$), recursively building branches, and creating an interpretable model that predicts tennis playing decisions.

Key insight: Decision trees transform complex datasets into simple, explainable rules through systematic mathematical optimization.

Your Next Steps

1

Practice Calculations

Work through entropy, IG, and Gini on paper with sample datasets to build intuition

2

Code Implementation

Build decision trees using Python's sklearn with the Play Tennis dataset

3

Explore Advanced Techniques

Learn pruning strategies and ensemble methods (Random Forests, Gradient Boosting)

4

Apply Wisely

Remember limitations and choose appropriate algorithms for your specific problem

"Decision trees are powerful tools for interpretable machine learning, but like any algorithm, they shine brightest when you understand both their strengths and limitations."

You now have the mathematical foundation and practical intuition to build, optimize, and critically evaluate decision tree models. Happy learning!



DATA SCIENCE LEARNING PATH